

REFACTORIA: A Game-Based Tool for Learning Code Refactoring

Abstract

Refactoring is a crucial practice in software maintenance, that aims at improving coding practices while addressing design flaws. Due to the importance of the topic, several methods and strategies have been proposed for teaching and training refactoring, including interactive learning, incremental learning, gamification, visualization, and experimentation. However, the skills needed to identify and refactor code smells come from training and experience, and the lack of motivation may hinder developers in the adoption of refactoring tools. In this paper, we aim at developing REFACTORIA, an innovative game-based learning environment that supports the acquisition of code smell and refactoring concepts through interactive gameplay. We believe that REFACTORIA contributes a unique blend of pedagogical innovation and practical relevance. The source code is publicly available on GitHub at <https://github.com/SE4AIRResearch/RefactorGame>. The demonstration video is available at <https://youtu.be/VOJcB3qcLJY>.

CCS Concepts

• **Software and its engineering** → **Software maintenance tools**.

Keywords

refactoring, code smell, game, software quality

1 Introduction

Refactoring is the art of remodeling the software design without changing its external behavior [10]. When performed effectively, it leads to significant software quality improvement. Thus, refactoring is a recommended practice according to the software engineering body of knowledge [7]. Being critical to software quality, refactoring research has been of importance to practitioners, researchers, and educators. Despite the fact that refactoring is considered useful and important, surveys have shown a lack of adoption of refactoring tools in practice [15, 18], highlighting the importance of teaching software engineering tools to students [17].

Refactoring is a critical software maintenance activity that developers perform for a number of reasons [5, 18]. One of the main refactoring motivations is code smell resolution. Code smell is any characteristic of software that might indicate a problem, negatively affecting software quality [10]. The skills needed to identify and refactor code smells come from training and experience [8, 13], and

lack of motivation can hinder developers in the adoption of refactoring tools [9]. Although these advantages exist, skills in code smells and refactoring have been neglected in some computer science or software engineering courses [8], as these topics are generally less popular compared to design patterns. In many computer science or software engineering curricula, instructors highlight software quality when teaching good programming practices and design patterns. However, it is necessary to reveal how deviation from best practices can lead to poor design choices that negatively impact the source code [4]. Due to the importance of the topic, several methods and strategies for teaching and training refactoring have been proposed including interactive learning [4, 12], incremental learning [19, 20], gamification [3, 8, 9, 13], visualization [16], experimentation [1, 4], and e-activity planning [14]. Although these methods enhance a student's grasp of refactoring, it is essential to introduce students to more profound design-level antipatterns, which often appear even in the most well-engineered projects and are more challenging to resolve.

To cope with the above-mentioned challenges, we developed REFACTORIA, a novel game-based learning environment that teaches the basic concepts of refactoring. Therefore, the learning goals of REFACTORIA are related to growing as a programmer by understanding indicators of poor code design and how to fix the underlying issues that those indicators represent. The overall gameplay revolves around editing code; more specifically, the players are given flawed code and told to improve it. This gameplay loop should be easily recontextualized in the real world, as players reapply the lessons they learn in external situations. The narrative, puzzle structure, and learning feedback all contribute to this core learning concept.

The players learn to identify various code smells and apply refactoring techniques to eliminate them. REFACTORIA follows a chef and their cooking robot, Watson the Whiskbot, in their quest to improve Watson's code base. On each of the thirteen levels, the player is given some code that, when run, accepts a list of orders and directs Watson to cook and deliver each order. Although the code passes the test cases, it is riddled with code smells. The player is then tasked to refactor the code to eliminate the code smells using refactoring techniques while still passing all test cases. At the end of each level, the player is presented with the names of each refactoring technique they used and each code smell that they discovered. Each technique and smell are added to the game dictionary after discovery so that players can reference previously discovered techniques and smells. Techniques and smells build on each other throughout the game. The main contributions of this paper are summarized as follows:

- We propose REFACTORIA, a novel and innovative game-based learning environment that supports the acquisition of code smell and refactoring concepts through interactive gameplay.
- We design our custom language called, RefactorLang, consisting of tokenizer-compiler-interpreter which is then integrated with Unity¹ to drive game animations.

* These authors contributed equally.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2026 ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnnn.nnnnnnn>

¹<https://unity.com/>

- We release our game, and source code as open source for reproducibility and extension².

2 Related Work

Santos *et al.* [8] implemented CLEANGAME, a gamified tool that helps to identify five code smells. The game consists of two modules: (1) smell-related quiz (*i.e.*, shows questions about code smell resolution with multiple-choice answers), and (2) code smell identification. Their findings reveal that participants managed to identify twice as many code smells with a gamified approach compared to a non-gamified approach. Haendler [12] proposed a non-digital multi-player card game called REFACTORY to learn the principles of refactoring without development. Their user study indicates that the card game can encourage learning about refactoring and support learning refactoring principles. Elezi *et al.* [9] created CODEARENA, a gamification system that tracks and rewards refactorings during development. While useful, the authors indicated that the game does not perform any quality evaluation on the types refactorings completed, and does not evaluate if the refactored code still works by passing test cases. Baars and Meester [6] proposed CODEARENA, an extension to the popular 3D sandbox game called MINECRAFT [11], that converts patterns in a codebase that are considered harmful for the purpose of improving code quality. When the Java project has been selected, the tool creates a monster for each code issue found. The user then needs to fix code issues to collect points. Haendler and Neumann [13] investigated the possibilities and challenges of designing serious games for refactoring on real-world code artifacts by proposing a game design, where students can compete either against a predefined benchmark (technical debt) or against each other. Agrahari and Chimalakonda [2] developed REFACTOR4GREEN, a desktop game created using the Unity 3D game engine to teach code smells and refactoring to novice programmers. The central concept of the game involves incorporating code smells via refactoring decisions, all within the theme of an environmentally green setting. The authors focused on code smells that result in more energy consumption of software. Aljedaani *et al.* [3] proposed REFGAME to enhance the learning experience by identifying and applying refactoring techniques. The game consists of three different types: matching, racing, and snake games, each highlights on different aspects of refactoring.

3 Game Design

In the narrative of REFACTORIA, the player has recently gained a restaurant, complete with a robotic sous chef, Watson the Whiskbot (See Figure 1). The recipes for different food items are represented by different programs for Watson. The legacy recipes implemented by the restaurant's previous owner are called out to be suboptimal, though, so it falls upon the player to make the necessary changes to the programs. This is reinforced by the game mechanic of constraints: the provided code must be limited to some number of lines, or the number of parameters used must be less than a given amount, and so on.

4 REFACTORIA Puzzles

4.1 Welcome to REFACTORIA

"Welcome to REFACTORIA" is the first level of the game, and it also serves as a tutorial. The story prompt welcomes the player and explicitly tells them that they must refactor the code to shorten it. Here, we are less concerned with discussing "Duplicate Code". We spend this time exemplifying to the player how our custom language works and what all of the game objects do. Various tooltips appear on the screen to explain the function of the game icons. The program shows the player that Watson must move around the kitchen to perform certain actions. Watson gets food from the pantry, delivers food to the window, etc. The solution to this puzzle is to remove the two clearly extraneous movement commands from the program (bringing the line count down to the satisfying number). To make finding this extraneous code segment easier, it is separated by whitespace from the other parts of the code. This solution also gives players a frame of reference for the types of mistakes they should be looking for in future levels.

Code Smell: Duplicate Code

Constraint: Reduce the code to 13 lines or less.

Story Prompt: *Watson, how do you make tomato soup? No, not quite. Task: Find the issue with the code, and refactor it to make it shorter and clearer.*

4.2 One Soup Two Soup

"One Soup Two Soup" is another introductory puzzle for the player to get acquainted with our custom language and the game objects. There are two intentions behind this puzzle. First, this level introduces a program that must pass multiple test cases to the player and shows the player the syntax for taking the "food orders" as inputs. The second intention behind this puzzle is to show another, subtler example of "Duplicate Code". This is hinted at in the story prompt, which suggests that the program is "much larger than it needs to be". The initial program in this puzzle runs one of two large code segments depending on the result of an if-else conditional. Players should notice that these two code segments almost entirely consist of the same lines. The solution is to move these duplicate lines outside of the if-statement. Moving these lines will bring the program to under 16 lines, passing the level constraints.

Code Smell: Duplicate Code

Constraint: Reduce the code to 15 lines or less.

Story Prompt: *I ask Watson to make me some tomato soup. Next, potato soup. Right away, I see his instruction set is much larger than it needs to be. Tip: The same code must be used for all orders!*

4.3 Future Recipes

This puzzle is about "Speculative Generality", unused code that is said to be necessary for the future but bloats the code in the meantime. The code presented in this level does not inherently need to be refactored, but with context, the player can see that it is a problem. According to the story blurb, the old owner of the restaurant wanted to add squash soup to the menu, but he never did. The puzzle title "Future Recipes" indicates that this speculation is the main idea presented here. This is a clear example of the kind of speculation we want our players to be able to recognize as a

²<https://refactorings.github.io/education/>

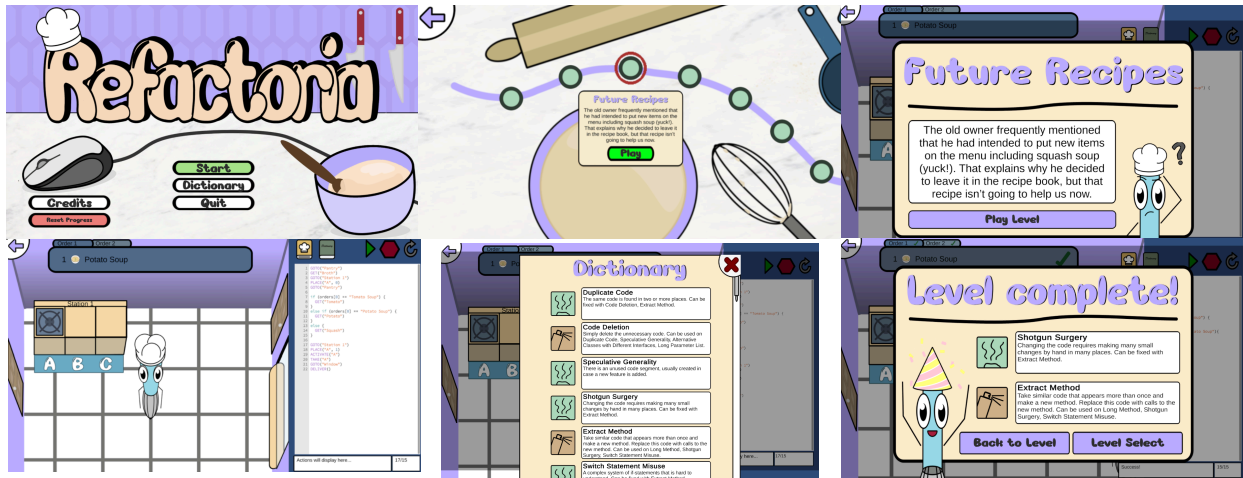


Figure 1: REFACTORIA graphical user interface in action, showing the “Future Recipes” puzzle.

problem in the code. To solve this puzzle, the player must notice that there is code that would be activated if Squash Soup is ordered, but Squash Soup is not included in the test cases. Therefore, the player must delete the code related to Squash Soup (successfully bringing down their line count to pass the constraint).

Code Smell: Speculative Generality

Constraint: Reduce the code to 15 lines or less.

Story Prompt: *The old owner frequently mentioned that he had intended to put new items on the menu including squash soup (yuck!). That explains why he decided to leave it in the recipe book, but that recipe isn't going to help us now.*

4.4 A Method to the Madness

In the next few puzzles, we increase their complexity. To prevent the player from being overwhelmed, we created this puzzle to introduce the syntax for defining methods, calling methods, using while-loops, and accessing the global variable #orders, which returns the number of items in the test case. The puzzle title implicitly tells the player to focus on the method structure because they will need it later. To solve this puzzle, the player must notice that the while loop is written with an extra method call. Cleaning up this while loop solves the puzzle by bringing the number of lines down to an acceptable number. By solving this puzzle, the player will have demonstrated that they understand the syntax of while-loops, which will be important later.

Code Smell: Duplicate Code

Constraint: Reduce the code to 16 lines or less.

Story Prompt: *A tomato soup craze sweeps the nation. Realizing I can't type fast enough to keep up, I search for a better solution.*

4.5 R&D: Soup on Soup

This is the first of three puzzles designed to teach the player about “Shotgun Surgery”. This trilogy is marked by the research and development storyline that ties these puzzles together and the label “R&D” in each puzzle’s title. The reason why this code smell is

taught across three puzzles is that we want the player to experience the negative effects of “Shotgun Surgery” firsthand and then work to fix it. These are the only three puzzles in the game where the player is given non-functioning code. For this first puzzle, the player must change every appearance of “Tomato” to “Mushroom”. In essence, they are engaging in the tedious process of “Shotgun Surgery”. In addition, to review concepts previously taught in the game, the player must find and remove some “Duplicate Code” to bring the length of the program down to meet constraints.

Code Smell: Shotgun Surgery

Constraint: Reduce the code to 15 lines or less.

Story Prompt: *I'm looking to test a new soup on the menu. How about mushroom? With a glance to the left and right, I scribble it onto the menu. Now, I just need to teach Watson how to make it.*

4.6 R&D: All Together Now

As a twist for the player, the starter code for this puzzle is whatever the player submitted for the previous puzzle. They must now adapt the code to work for four types of soup instead of two. Like in a real-life instance of fixing “Shotgun Surgery” problems, the player must adapt the existing poor code to accommodate new features. Players can solve this however they see fit while staying under the constraints.

Code Smell: Shotgun Surgery

Constraint: Functionality. Stay under 20 lines of code or less.

Story Prompt: *With customers finally pouring in, Watson is super nervous. Such cowardice has no place in a well-run kitchen: Watson needs to pull it together. With a click of embarrassment, he recovers and prepares to execute orders.*

4.7 R&D: One More Soup

In this third level in the trilogy, the player starts with the solution code from the last puzzle once again and must accommodate one additional soup test case. If their solution to the previous puzzle was strong, then this puzzle will be easy. Due to the constraints,

the player's program in this puzzle must be much denser. The code must accommodate 5 test cases with only a few more lines to work with. As a result, the player will need to have developed a refactored solution to the problem (one that could handle one, two, or more new similar test cases with ease). By the end of this trilogy of "R&D" puzzles, the player will have experienced first-hand why "Shotgun Surgery" is tedious to deal with and will have developed their own solution to solve it.

Code Smell: Shotgun Surgery

Constraint: Stay under 23 lines of code or less.

Story Prompt: *Watson moves in a symphony of slicing and boiling. It nearly brings a tear to my eye. But our patrons insist on one more kind of soup. Can he handle it? I scratched down one idea, but we'll see...*

4.8 Take Two

The level name and story blurb in this level establish the following premise: All test cases are now pairs of soups instead of single soups. In this code, there is an uncomfortably long "Switch Statement". The player can plainly see that this makes the code harder to read and understand. The solution is to use "Extract Method" and replace the switch statement with a while loop. After the player's code meets the constraints, they will have demonstrated that they understand how to use the "Extract Method" technique.

Code Smell: Switch Statement Misuse

Constraint: All if-statements must have no more than one comparison. Maximum of 24 lines.

Story Prompt: *What do you do if someone orders two different kinds of soup? I ask. Watson's face tells me that he's feeling bluer than usual. Oh, boy! Extra requirement: If-statements must have no more than one comparison.*

4.9 Build a Burger

In "Build a Burger", the player is confronted with making a new food, burgers, and with four new stations: the Loaf Slicer, the Barbecue Saucer, the Grinder, and the Burger Builder. The code given to the player is straightforward but long. The story prompt suggests it is so long it "wraps around the kitchen". The main obstacle to understanding the code is that it is not broken down into sections. By forcing the player to break the code down into at least six methods, they will naturally learn what each of the stations does, and they will naturally refactor the main problem of the code.

Code Smell: Long Method

Constraint: Use at least six methods. Maximum of 32 lines.

Story Prompt: *The first time I ask Watson how he makes a burger, he prints out a main method so long it wrapped around the whole kitchen. Come now, we can do better than this. Extra requirement: Use at least six methods.*

4.10 All Over the Place

"All Over the Place" introduces the idea of moving modules around the kitchen with a simple example, using a new recipe and the new station, the Fryer. At the start of the puzzle, the modules are spread across the kitchen, which forces Watson to take additional move actions to get from place to place. The player must move all three of the modules to the same station to remove the need for this

move action. This demonstrates the core motivation behind fixing "Feature Envy": similar tasks should be done close to each other. After solving this problem, Watson will not need to move as much, so the number of movement statements can be reduced, and the total length of the program can be reduced to meet the constraint.

Code Smell: Feature Envy

Constraint: Reduce the code to 19 lines or less.

Story Prompt: *Today, Watson, in a rush to cross the kitchen, knocked me onto the floor. After staggering to my feet, I realize I haven't been taking advantage of Refactoria's highly configurable kitchen modules. Tip: Click on a station's module to change it.*

4.11 A Tight Shift

Both the level title and the story prompt clue that this puzzle is about large-scale redundancy. This puzzle revolves around spotting duplicate classes. At the start of this puzzle, there are two stations with an identical set of modules. The constraint placed on the player is that they must use the minimum number of total modules necessary. The solution is simply to delete all the duplicate modules and make sure that all the tasks are being done at a single station. To shorten the code, the player may reduce the `prep_bread()` and `prep_chicken()` methods to one `prep_ingredient()` method with parameters, because much of the code in the original two methods is the same. This will minimize the length of the code to meet the constraints and demonstrate understanding of "Add Parameter".

Code Smell: Alternative Classes with Different Interfaces

Constraint: The kitchen must have no more than 3 modules. Maximum of 20 lines.

Story Prompt: *My heart sinks when I see the energy bill at the end of the month. The horror! And then I find the recipe responsible for this bill... Extra requirement: Don't use more modules than necessary.*

4.12 Parameter Overload

As clued by the title and story blurb, this puzzle is about methods with long lists of parameters. The player is given an updated version of the program that makes chicken tenders. This code is shorter due to its use of parameters, but many of these parameters connect to variables that are only used once. These parameters do not shorten the code and only hinder understanding of how the code works. The constraint of this puzzle is that players must use the minimum number of total parameters needed to keep the code as efficient as it currently is. Once players remove all but three of the parameters, they will have solved the puzzle. They will also have demonstrated that they can determine when using a parameter does not serve the program.

Code Smell: Long Parameter List

Constraint: The program cannot contain more than 3 parameters. Maximum of 14 lines.

Story Prompt: *The old chef went too far with parameterizing this recipe. Perhaps I can salvage it. Extra requirement: Don't use more total method parameters in the code than necessary.*

4.13 An Industrial Kitchen

This final puzzle in the game is also the hardest. The story prompt clues that the kitchen is disorganized. The player is given code that creates both burgers and chicken tenders, depending on what is

ordered. The code is fully functional, but the way the modules are organized in their stations makes no sense. For example, the only two modules that create burger toppings are located in different stations. This is an analog for a “Large Class”. The constraint placed on the player is a maximum of two modules per station instead of the usual three. The player must refactor the code so that the set of stations is organized more meaningfully. Once they have done that, they will have demonstrated knowledge of “Extract Class”.

Code Smell: Large Class

Constraint: Maximum of 2 modules per station.

Story Prompt: *This is it, we're nearly back to full service. I'm proud of Watson. Without his hard work, we never would have reached this point. I survey the kitchen and realize that he has an awful lot on his plate. Extra requirement: Use a maximum of 2 modules per station.*

5 Limitations and future work

The renaming refactoring concept is one of the easiest to understand, both in why the code smell associated with it is a problem and how to go about fixing it. The renaming puzzle we initially designed centered around renaming the names of stations and the names of functions in the provided starter code. This puzzle concept was eliminated for several reasons: (1) the action of renaming of stations was specific to this puzzle, which counteracts the “sand castle” approach of puzzle design that we were aiming for, (2) the constraint of limiting code to a specific number of statements counteracted the idea of renaming existing functions, and (3) it is challenging to verify that the new names entered by the player were good names to use in context; the most we could do was verify that the player had changed the names at all, potentially tempting them to rename them arbitrarily and dodging the point of the puzzle.

Another limitation central to the learning and gameplay of REFACTORIA was the use of modules in kitchen stations as an allegory for real classes. Our custom language is simple and only covers the basics. It does not contain classes as the inclusion of real classes was deemed to complex for the scale of the project. In our game, the stations “represent” classes and the modules within the stations “represent” methods. This is a good strategy for understanding code smells where class structure is irrelevant such as “Speculative Generality”. However, many refactoring techniques are meant for object-oriented code. The analogies used to teach the meaning of “Large Class”, “Extract Method” etc. were likely inefficient in many cases. Additionally, there were certain code smells and refactoring techniques we could not build puzzles around because of the limited scope and functionality of the kitchen station analogy. For example, in our system, we cannot design a puzzle around the techniques, “Push Down”/ “Pull Up” Field or “Push Down”/ “Pull Up” Method. A more advanced custom language will surely open the possibilities for refactoring-based puzzles as the code will be able to more closely mirror real-world programs.

References

- [1] Shamsa Abid, Hamid Abdul Basit, and Naveed Arshad. 2015. Reflections on teaching refactoring: A tale of two projects. In *Proceedings of the 2015 ACM Conference on Innovation and Technology in Computer Science Education*. 225–230.
- [2] Vartika Agrahari and Sridhar Chimalakonda. 2020. Refactor4green: A game for novice programmers to learn code smells. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings*. 324–325.
- [3] Wajdi Aljedaani, Anwar Ghammam, Mohamed Wiem Mkaouer, and Marouane Kessentini. 2024. From Boring to Boarding: Transforming Refactoring Education with Game-Based Learning. In *Proceedings of the ACM/IEEE 8th International Workshop on Games and Software Engineering*. 20–27.
- [4] Eman Abdullah AlOmar, Mohamed Wiem Mkaouer, and Ali Ouni. 2024. Automating source code refactoring in the classroom. In *Proceedings of the 55th ACM Technical Symposium on Computer Science Education V. 1*. 60–66.
- [5] Eman Abdullah AlOmar, Anthony Peruma, Mohamed Wiem Mkaouer, Christian Newman, Ali Ouni, and Marouane Kessentini. 2021. How we refactor and how we document it? On the use of supervised machine learning algorithms to classify refactoring documentation. *Expert Systems with Applications* 167 (2021), 114176.
- [6] Simon Baars and Sander Meester. 2019. Codearena: Inspecting and improving code quality metrics in java using minecraft. In *Proceedings of the 2019 International Conference on Technical Debt (Tool Demos)*. IEEE.
- [7] Pierre Bourque, Robert Dupuis, Alain Abran, James W Moore, and Leonard Tripp. 2002. The guide to the software engineering body of knowledge. *IEEE software* 16, 6 (2002), 35–44.
- [8] Hoyama Maria dos Santos, Vinicius HS Durelli, Maurício Souza, Eduardo Figueiredo, Lucas Timoteo da Silva, and Rafael S Durelli. 2019. Cleangame: Gamifying the identification of code smells. In *Proceedings of the XXXIII Brazilian Symposium on Software Engineering*. 437–446.
- [9] Leonard Elezi, Sara Sali, Serge Demeyer, Alessandro Murgia, and Javier Pérez. 2016. A game of refactoring: Studying the impact of gamification in software refactoring. In *Proceedings of the Scientific Workshop Proceedings of XP2016*. 1–6.
- [10] Martin Fowler, Kent Beck, John Brant, William Opdyke, and don Roberts. 1999. *Refactoring: Improving the Design of Existing Code*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA. <http://dl.acm.org/citation.cfm?id=311424>
- [11] Arun Gupta and Aditya Gupta. 2015. *Minecraft Modding with Forge: A Family-Friendly Guide to Building Fun Mods in Java*. O'Reilly Media, Inc.
- [12] Thorsten Haendler. 2019. A Card Game for Learning Software-Refactoring Principles.. In *GamiLearn*.
- [13] Thorsten Haendler and Gustaf Neumann. 2019. Serious refactoring games. In *Proceedings of the 52nd Hawaii International Conference on System Sciences*.
- [14] Carlos López, Jesús M Alonso, Raúl Marticorena, and Jesús M Maudes. 2014. Design of e-activities for the learning of code refactoring tasks. In *2014 International Symposium on Computers in Education (SIEE)*. IEEE, 35–40.
- [15] Emerson Murphy-Hill, Chris Parnin, and Andrew P Black. 2012. How We Refactor, and How We Know It. *IEEE Transactions on Software Engineering* 38, 1 (Jan 2012), 5–18. doi:10.1109/TSE.2011.41
- [16] Felix Raab. 2012. CodeSmellExplorer: Tangible exploration of code smells and refactorings. In *2012 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, 261–262.
- [17] Claudia Raibulet, Francesca Arcelli Fontana, and Ilaria Pigazzini. 2019. Teaching Software Engineering Tools to Undergraduate Students. In *Proceedings of the 2019 11th International Conference on Education Technology and Computers*. 262–267.
- [18] Danilo Silva, Nikolaos Tsantalos, and Marco Tulio Valente. 2016. Why We Refactor? Confessions of GitHub Contributors. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (Seattle, WA, USA) (FSE 2016)*. ACM, New York, NY, USA, 858–870. doi:10.1145/2950290.2950305
- [19] Suzanne Smith, Sara Stoecklin, and Catharina Serino. 2006. An innovative approach to teaching refactoring. In *Proceedings of the 37th SIGCSE technical symposium on Computer science education*. 349–353.
- [20] Sara Stoecklin, Suzanne Smith, and Catharina Serino. 2007. Teaching students to build well formed object-oriented methods through refactoring. *ACM SIGCSE Bulletin* 39, 1 (2007), 145–149.